ДИСЦИПЛИНА «Разработка программных средств поиска и анализа киберугроз»

ИНСТИТУТ «Институт кибербезопасности и цифровых технологий»

КАФЕДРА КБ-2 «Информационно-аналитические системы кибербезопасности»

ВИД УЧЕБНОГО МАТЕРИАЛА Лекция

ПРЕПОДАВАТЕЛЬ Новиков Евгений Иванович

СЕМЕСТР 9 семестр 2026-2027 учебный год

ЛЕКЦИЯ 6. КОДИРОВАНИЕ ПРИЗНАКОВ В КИБЕРБЕЗОПАСНОСТИ

План лекции:

1. Кодирование номинальных признаков.
2. Кодирование порядковых признаков.

In [ ]:
```
import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
import random
%config InlineBackend.figure_format = 'retina'
import scipy.stats as sts
pd.set_option('display.max_columns', None)
```

Кодирование признаков в кибербезопасности

Большинство алгоритмов машинного обучения работают с числовыми признаками: векторы, расстояния, скалярные произведения, градиентный спуск, регуляризация — все это требует числового представления.

При этом в данных по кибербезопасности достаточно часто используются категориальные признаки:

- `protocol`: `TCP`, `UDP`, `ICMP`;
- `service`: `ssh`, `http`, `dns`, `smb`, `rdp`;
- `country`: страна источника или назначения;
- `src_ip`, `dst_ip`: IP-адреса;
- `user_agent` (тип клиентского ПО), `process_name` (имя исполняемого процесса), `file_hash` (хеш файла), `url` (полный URL запроса), `domain` (доменное имя);
- `alert_rule` (название правила, по которому сгенерирован алерт), `mitre_technique` (техника MITRE ATT&CK), `event_type` (тип события в терминах источника).

Особенности кодирования в кибербезопасности.

В задачах кибербезпасности есть несколько важных особенностей, которые напрямую влияют на выбор метода кодирования.

1. Высокая кардинальность.

Многие категориальные признаки имеют огромное число значений:

- IP-адреса;
- домены;
- URL;
- хэши файлов;
- идентификаторы сессий;
- user-agent;
- имена процессов.

Если применить неправильный метод кодирования, можно получить миллионы признаков. Это приводит к разреженности, перерасходу памяти и ухудшению обобщающей способности модели.

2. Постоянное появление новых категорий.

В реальных условиях постоянно появляются новые категории признаков (например, новые домены). Поэтому кодировщик должен уметь обрабатывать неизвестные категории. Например:

- присваивать отдельное значение `unknown`;
- игнорировать неизвестную категорию;
- кодировать ее априорной статистикой.

3. Редкие категории и аномалии.

В кибербезопасности редкая категория часто может быть признаком атаки:

- редкий протокол;
- необычный сервис на нестандартном порту;
- редкое имя процесса;
- редкая страна источника;
- редкое DNS-имя.

Но редкость также может быть шумом или ошибкой телеметрии. Поэтому методы, основанные на частоте или целевой статистике, нужно сглаживать и ограничивать.

4. Утечка целевой переменной.

Если для кодирования используется статистика целевого признака, например, доля атак для страны или IP, то модель начнет использовать информацию из целевой переменной напрямую.

5. Концептуальный дрейф.

Поведение защищаемой системы постоянно меняется:

- появляются новые сервисы;
- меняются правила firewall;
- меняется профиль пользователей;
- изменяется география трафика;
- атакующие меняют тактику.

Статистики, рассчитанные месяц назад, могут устареть. Кодировщики нужно периодически переобучать или обновлять в режиме реального времени.

6. Интерпретируемость и расследование инцидентов.

В кибербезопасности часто важно не только предсказать атаку, но и объяснить решение:

- почему хост помечен как скомпрометированный;
- почему трафик считается подозрительным;
- какой признак внес наибольший вклад;
- какие категории связаны с риском.

В этом случае нужно выбирать кодировщики, которые являются интерпретируемыми.

В лекции будем разделять методы кодирования на две большие группы:

1. Кодирование номинальных признаков — категории без естественного порядка (например, `protocol`, `country`, `service`, `src_ip`).
2. Кодирование порядковых признаков — категории с порядком (например, `severity`: `low → medium → high → critical`).

Для изучения технологий кодирования категориальных признаков будем использовать набор данных, содержащихся в логах событий безопасности.

In [ ]:
```
# @title
data = {
    "event_id": list(range(1, 13)),
    "protocol": [
        "TCP", "TCP", "UDP", "TCP", "UDP", "ICMP",
        "TCP", "UDP", "TCP", "UDP", "TCP", "ICMP"
    ],
    "service": [
        "ssh", "http", "dns", "smb", "rdp", "none",
        "http", "dns", "ssh", "dhcp", "http", "none"
    ],
    "country": [
        "CN", "US", "RU", "CN", "KP", "CN",
        "IR", "US", "RU", "RU", "US", "US"
    ],
    "alert_rule": [
        "SSH brute force", "Web normal", "DNS normal", "SMB lateral",
        "RDP brute force", "ICMP scan", "SQL injection", "DNS normal",
        "SSH normal", "DHCP discover", "Web scan", "ICMP normal"
    ],
    "src_ip": [
        "10.0.0.1", "192.168.1.10", "10.0.0.2", "10.0.0.1",
        "203.0.113.5", "203.0.113.9", "198.51.100.7", "192.168.1.11",
        "10.0.0.3", "10.0.0.4", "198.51.100.7", "192.168.1.12"
    ],
    "is_attack": [1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0]
}

df = pd.DataFrame(data)
cat_cols = ["protocol", "service", "country", "alert_rule", "src_ip"]
df = df.sort_values("event_id").reset_index(drop=True)
```

In [ ]:
```
df.head()
```

Out[ ]:

	event_id	protocol	service	country	alert_rule	src_ip	is_attack
0	1	TCP	ssh	CN	SSH brute force	10.0.0.1	1
1	2	TCP	http	US	Web normal	192.168.1.10	0
2	3	UDP	dns	RU	DNS normal	10.0.0.2	0
3	4	TCP	smb	CN	SMB lateral	10.0.0.1	1
4	5	UDP	rdp	KP	RDP brute force	203.0.113.5	1

Признаки:

- `event_id` : уникальный идентификатор типа события в системе;
- `protocol` : сетевой протокол транспортного уровня;
- `service` : прикладная служба, к которой относится событие;
- `country` : страна источника;
- `alert_rule` : имя правила, по которому сработало оповещение;
- `src_ip` : IP-адрес источника события;
- `is_attack` : целевая переменная - бинарный индикатор, было ли событие частью атаки(0 - норма, 1 - атака).

1. Кодирование номинальных признаков

Для номинальных признаков категории не могут быть проранжированы, и нет логической возможности их порядкового сравнения между собой, например:

- протокол: `TCP` , `UDP` , `ICMP` ;
- сервис: `ssh` , `http` , `dns` , `smb` ;
- страна: `RU` , `US` , `CN` , `IR` .

Основные методы кодирования номинальных признаков:

- Label Encoding;
- One-Hot Encoding;
- Dummy Encoding;
- Binary Encoding;
- Feature Hashing / hashing trick;
- Count / Frequency Encoding;
- Target / Mean Encoding;.
- Leave-One-Out Target Encoding.
- Ordered Target Encoding / CatBoost-подобное кодирование.

Большинство из перечисленных методов кодирования реализованы в библиотеках `pandas` и `scikit-learn` . Так же созданы специальные библиотеки для

реализации более широкого набора методов, например библиотека `category_encoders`, которая полностью совместима с `scikit-learn`.

Label Encoding

Этот метод кодирования является наиболее простыми и часто используемыми. Кодирование заключается в присвоении каждому уникальному значению категориального признака целого числа.

Первое (выбранное каким-то образом) уникальное значение признака кодируется нулем, второе единицей, и так далее, последнее кодируется числом, равным количеству уникальных значений минус один.

Формально признак становится числовым, но число не несет физического смысла величины.

Например, при кодировании признака `protocol` было получено:

- `ICMP` - 0;
- `TCP` - 1;
- `UDP` - 2.

Код 2 для `UDP` не означает, что `UDP` «в два раза больше» `TCP`.

Ограничения метода:

1. Создает искусственный порядок. Модель может воспринять код 0 < 1 < 2 как реальный порядок.
2. Плохо подходит для линейных моделей. Линейная регрессия, логистическая регрессия, SVM с линейным ядром могут неправильно интерпретировать такие числа.
3. Плохо подходит для метрических моделей. kNN и методы, использующие расстояния, могут считать, что расстояние между ICMP и TCP равно 1, а между `ICMP` и `UDP` равно 2, что лишено смысла.
4. Кодирование зависит от сортировки. Если изменить порядок категорий, изменятся коды, но смысл признака не изменится.
5. Не решает проблему новых категорий. Нужно отдельно хранить словарь и решать, что делать с неизвестной категорией.

Рекомендации по использованию в кибербезопасности.

Label Encoding можно использовать, если:

- признак имеет низкую или среднюю кардинальность;
- далее будет использоваться модель на основе дерева решений: Decision Tree, Random Forest, Gradient Boosting;
- категория является идентификатором, а не смысловой величиной.

Не рекомендуется использовать:

- для линейных моделей;
- для метрических моделей;
- для высококардинальных признаков;
- если важна интерпретируемость.

В кибербезопасности Label Encoding допустим как простой baseline, но часто лучше выбрать one-hot, hashing или target-based методы.

Для реализации этого кодировщика можно использовать несколько подходов:

- "ручное" кодирование на основе механизма `mapping`;
- преобразование признака к типу `'category'` и использование метода `cat.codes()`;
- использование классов `LabelEncoder` или `OrdinalEncoder` модуля `preprocessing` библиотеки `sklearn`;
- использование класса `OrdinalEncoder` библиотеки `category_encoders`.

Рассмотрим пример использования этих кодировщиков для кодирования признака `protocol`.

mapping

Этот способ позволяет задать произвольную схему кодирования значений признака, передать ее в виде словаря сопоставления (mapping) в функцию `map` или `replace` и применить к соответствующему признаку.

In [ ]:
```
df_le = df.copy()
protocol_codes = {
    category: code
    for code, category in enumerate(df_le["protocol"].unique())
}

df_le["protocol_label_map"] = df_le["protocol"].map(protocol_codes)
df_le[["protocol", "protocol_label_map"]].head(7)
```

Out[ ]:

	protocol	protocol_label_map
0	TCP	0
1	TCP	0
2	UDP	1
3	TCP	0
4	UDP	1
5	ICMP	2
6	TCP	0

cat.codes

Этот подход основан на преобразовании качественных признаков к типу `'category'` и применении к ним атрибута `DataFrame.cat.codes` либо `Series.cat.codes`.

При этом перевод категориальных признаков в тип `category` часто позволяет повысить производительность и уменьшить использование памяти.

Для приведения данных к типу `'category'` можно использовать метод `astype`.

При создании категориального признака каждой категории признака автоматически присваивается целочисленное значение - код (code), который можно просмотреть с помощью свойства `cat.codes`.

Можно увидеть, что создаваемые категории сначала сортируются в алфавитном порядке, а потом им присваиваются соответствующие коды.

In [ ]:
```
df_le['protocol_cat'] = df_le['protocol'].astype('category')
df_le['protocol_label_cat'] = df_le['protocol_cat'].cat.codes
df_le[["protocol", "protocol_label_map", "protocol_label_cat"]].head(7)
```

Out[ ]:

	protocol	protocol_label_map	protocol_label_cat
0	TCP	0	1
1	TCP	0	1
2	UDP	1	2
3	TCP	0	1
4	UDP	1	2
5	ICMP	2	0
6	TCP	0	1

Sklearn

В библиотеке `sklearn` для реализации метода Label Encoding можно использовать классы `LabelEncoder` или `OrdinalEncoder` модуля `preprocessing`.

Класс `LabelEncoder` предназначен для кодирования целевого признака. Поэтому на вход этого класса поступают данные, имеющие форму `n_samples` (одномерный массив). У `LabelEncoder` нет настраиваемых параметров - это простой трансформатор, который присваивает каждой уникальной категории в целевом признаке уникальное целое число от 0 до n_classes-1.

`OrdinalEncoder` предназначен для кодирования независимых признаков, поэтому принимает на вход данные, имеющие форму (`n_samples` , `n_features`) (двумерный массив).

Основные параметры:

- `categories` (по умолчанию `auto`): задаёт порядок категорий для кодирования:
  - `auto`: категории определяются по данным, а порядок - лексикографический;
  - `list : categories[i]`: содержит категории, ожидаемые в i-м столбце. Позволяет задать правильный порядок категорий (для номинальных признаков не используется);
- `dtype` (по умолчанию `numpy.float64`): тип данных на выходе. Чаще всего оставляют как есть; для экономии памяти в больших пайплайнах можно ставить `int32` или `int64`, если не нужна дробность;
- `handle_unknown` (по умолчанию `error`): как поступать с категориями, которых не было в обучении:
  - error - при преобразовании будет выдана ошибка, если присутствует неизвестная категория;
  - `use_encoded_value` (по умолчанию `error`): закодированное значение неизвестных категорий будет соответствовать значению, указанному в параметре `unknown_value` (в задачах кибербезопасности, где появляются новые типы атак, часто выбирают `use_encoded_value` и задают понятный код (например, -1));
- `unknown_value` (по умолчанию `None`): может быть `int` или `np.nan`. Если для параметра `handle_unknown = use_encoded_value`, этот параметр является обязательным и определяет каким значением кодировать неизвестные категории;
- `encoded_missing_value` (по умолчанию `np.nan`): каким значением кодировать пропуски;
- `min_frequency` (по умолчанию `None`): указывает минимальную частоту, при которой категория будет считаться редкой;
- `max_categories` (по умолчанию `None`): максимальное число категорий. Все категории сверх лимита попадают в одну «редкую» группу (считается частота каждой категории в обучающей выборке, выбираются топ-max_categories категорий по частоте, все остальные категории попадают в группу «редких»).

In [ ]:
```
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder()
df_le['protocol_label_sklearn'] = oe.fit_transform(df_le[['protocol']])
df_le[["protocol", "protocol_label_map", "protocol_label_cat",
       "protocol_label_sklearn"]].head(7)
```

Out[ ]:

	protocol	protocol_label_map	protocol_label_cat	protocol_label_sklearn
0	TCP	0	1	1.0
1	TCP	0	1	1.0
2	UDP	1	2	2.0
3	TCP	0	1	1.0
4	UDP	1	2	2.0
5	ICMP	2	0	0.0
6	TCP	0	1	1.0

Посмотрим на правило преобразования с помощью атрибута `classes_`, который позволяет вывести категории признака.

In [ ]:
```
oe.categories_
```

Out[ ]:
```
[array(['ICMP', 'TCP', 'UDP'], dtype=object)]
```

Можно увидеть, что реализация метода Label Encoding в `sklearn` прежде всего сортирует по алфавиту уникальные значения, а потом присваивает им порядковый номер.

Category Encoders

Альтернативой рассмотренных средств кодирования может выступать библиотека `Category Encoders`, в которой реализовано несколько десятков различных методов кодирования. При этом она полностью совместима с библиотекой `Sklearn`.

Для установки библиотеки необходимо выполнить следующую команду.

In [ ]:
```
# @title
!pip install category_encoders
```

```
Collecting category_encoders
  Downloading category_encoders-2.11.1-py3-none-any.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.13/dist-pa
ckages (from category_encoders) (2.1.3)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.13/dist-pa
ckages (from category_encoders) (2.2.3)
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.13/d
ist-packages (from category_encoders) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.13/dist-pac
kages (from category_encoders) (1.16.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.1
3/dist-packages (from pandas>=1.0.5->category_encoders) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-pac
kages (from pandas>=1.0.5->category_encoders) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-p
ackages (from pandas>=1.0.5->category_encoders) (2026.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-pa
ckages (from scikit-learn>=1.6.0->category_encoders) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/
dist-packages (from scikit-learn>=1.6.0->category_encoders) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist
-packages (from joblib>=1.2.0->scikit-learn>=1.6.0->category_encoders) (3.1.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-package
s (from python-dateutil>=2.8.2->pandas>=1.0.5->category_encoders) (1.17.0)
Downloading category_encoders-2.11.1-py3-none-any.whl (107 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 107.9/107.9 kB 499.1 kB/s eta 0:00:00
Installing collected packages: category_encoders
Successfully installed category_encoders-2.11.1
```

После чего можно импортировать библиотеку в среду разработки.

Для кодирования методом Label Encoding используется класс `OrdinalEncoder` модуля `ordinal` библиотеки `category_encoders`.

Основные параметры:

- `cols` (по умолчанию `None`): список признаков для кодирования. Если не задан, кодируются все колонки с типами `object` или `category`;
- `mapping` (по умолчанию `None`): способ задать порядок категорий. Можно передать список словарей вида [{ 'col': 'country', 'mapping': {'RU': 0, 'CN': 1, 'US': 2} }]. Тогда кодирование будет строго по этим числам. Если `mapping=None`, коды назначаются в порядке первого появления категорий в обучающих данных, начиная с 1;
- `return_df` (по умолчанию `True`): если `True`, возвращается `pandas.DataFrame` с теми же именами колонок; если `False` - `numpy.ndarray`;
- `handle_unknown` (по умолчанию `value`): что делать с категориями, которых не было в обучении:
  - `value`: кодировать как `unknown_value`;
  - `return_nan`: оставить `NaN`;
  - `error`: выдать ошибку;
- `unknown_value` (по умолчанию -1): значение для неизвестных категорий (работает при `handle_unknown=value`);
- `handle_missing` (по умолчанию `value`) — как обрабатывать пропуски:

    - `value` : заменить на `missing_value` ;
    - `return_nan` : оставить `NaN` ;
    - `error` : выдать ошибку;
    - `indicator` : пропущенные значения будут рассматриваться как отдельная категория, а в обучающем наборе будет добавлен дополнительный фиктивный столбец (независимо от того, есть ли в обучающем наборе пропущенные значения);
- `missing_value` (по умолчанию -1): значение, которым кодируются пропуски при `handle_missing=value` ;
- `index_start` (по умолчанию 1): целое число, с которого начинается кодирование признакаужно начинать помечать категории.

In [ ]:
```
import category_encoders as ce
ce = ce.ordinal.OrdinalEncoder()
df_le['protocol_label_ce'] = ce.fit_transform(df_le['protocol'])
df_le[["protocol", "protocol_label_map", "protocol_label_cat",
       "protocol_label_sklearn", "protocol_label_ce"]].head(7)
```

Out[ ]:

	protocol	protocol_label_map	protocol_label_cat	protocol_label_sklearn	protocol_labe
0	TCP	0	1	1.0	
1	TCP	0	1	1.0	
2	UDP	1	2	2.0	
3	TCP	0	1	1.0	
4	UDP	1	2	2.0	
5	ICMP	2	0	0.0	
6	TCP	0	1	1.0	

Метод Label Encoding можно использовать для номинальных признаков только в случае построения моделей машинного обучения, основанных на дереве решений: Decision Tree, Random Forest, Gradient Boosting.

Что будет, если закодированные этим методом признаки использовать в линейных или метрических моделях?

Линейные модели (Linear Regression, Logistic Regression).

Модель обучается искать веса w_i так, чтобы прогноз был $y = w_0 + w_1 \cdot x_1 + \dots$.

Если x_1 закодирована методом Label Encoding: `ICMP → 0` , `TCP → 1` , `UDP → 2` , то модель может решить, что эффект от перехода от `ICMP` к `UDP` в два раза сильнее, чем от `ICMP` к `TCP` , хотя никакой такой зависимости в реальности нет. Это приводит к:

- искажённым коэффициентам (весам);
- неправильной интерпретации важности признака;

- снижению качества предсказаний, потому что модель подгоняется под искусственный порядок.

kNN и кластеризация (k-Means, DBSCAN и т. п.).

Эти алгоритмы напрямую используют расстояния между наблюдениями (чаще всего евклидово).

Если признак `protocol` закодирован как `ICMP → 0`, `TCP → 1`, `UDP → 2`, то расстояние между `ICMP` и `UDP` будет равно 2, а между `ICMP` и `TCP` — 1. Модель будет считать, что `UDP` «дальше» от `ICMP`, чем `TCP`, хотя в природе протоколов такой метрики нет. В результате:

- объекты с разными протоколами будут группироваться по «числовой близости», а не по смыслу;
- кластеры могут разделиться или соединиться неправильно;
- соседи в kNN будут выбираться искажённо.

One-Hot Encoding

Основным методом кодирования номинальных признаков является One-Hot Encoding (унитарное кодирование, горячее кодирование, кодирование одним состоянием).

Этот метод основан на создании для каждой категории признака нового столбца - фиктивной переменной (dummy variable), который может содержать значения 0 или 1. Единица означает наличие категории, а ноль — её отсутствие.

Ограничения метода:

1. Взрывной рост размерности при высокой кардинальности.
2. Разреженность. Если категорий много, матрица становится сильно разреженной.
3. Проблема мультиколлинеарности. Для линейных моделей без регуляризации нужно удалять одну категорию.
4. Редкие категории. Категория, встретившаяся один раз, может привести к неустойчивой оценке.
5. Новые категории. Нужно явно определять стратегию: `unknown`, отдельный признак, игнорирование.

Рассмотрим пример использования One-Hot Encoding для кодирования признака `protocol`.

Sklearn

Реализовать метод кодирования One-Hot Encoding можно через класс `OneHotEncoder` модуля `preprocessing` библиотеки `Sklearn`.

Основные параметры:

- `categories` (по умолчанию `auto`): задаёт категории для кодирования:
  - `auto`: категории определяются по уникальным значениям в обучающей выборке;
  - `list : categories[i]`: список списков: можно явно задать порядок и состав категорий;
- `drop` (по умолчанию `None`): удаляет одну из категорий для каждой колонки, чтобы избежать полной мультиколлинеарности (важно для линейных моделей):
  - `None`: все категории остаются (размерность = число уникальных значений);
  - `first`: удаляется первая категория в порядке `categories`;
  - `last`: удаляется последняя в порядке `categories`;
  - `array`: можно указать конкретные категории для удаления;
- `sparse_output` (по умолчанию `True`): позволяет задать форму вывода массива с кодами:
  - `False` - вернёт обычный `numpy.ndarray` (плотная матрица);
  - `True` - вернёт разреженную матрицу (обычно формата CSR — Compressed Sparse Row) из библиотеки `scipy.sparse`;

В разреженной матрице хранятся только ненулевые значения (в случае One-Hot — это единицы) и их позиции, а все нули явно не хранятся. Для One-Hot это особенно эффективно: в каждой строке ровно одна единица, остальные — нули.

Пример: если у нас 100 000 строк и 10 категорий, плотная матрица будет занимать место под 100 000 × 10 = 1 000 000 чисел. В разреженной — только 100 000 единиц плюс метаданные о позициях. Экономия памяти может быть в разы или даже на порядки.

Разреженные матрицы можно передавать в большинство моделей библиотеки `scikit-learn`: `LogisticRegression`, `LinearSVC`, `SGDClassifier`, `RandomForestClassifier` и т.д.

- `handle_unknown` (по умолчанию `error`): задает стратегию работы с новыми категориями:
  - `error`: выдается ошибка;
  - `ignore` - если в новых данных встретится категория, которой не было при обучении (например, новый протокол), это не вызовет ошибку, а просто для неизвестной категории все бинарные признаки будут 0 (вектор из нулей) - модель просто не «увидит» новую категорию;
  - `infrequent_if_exist`: при обнаружении неизвестной категории в результате преобразования все столбцы с однократным кодированием для этого признака будут соответствовать редко встречающейся категории, если таковая существует;
- `min_frequency` (по умолчанию `None`): указывает минимальную частоту, при которой категория будет считаться редкой. Можно задавать числом или долей, например 0.01 (1 % наблюдений);

- `max_categories` (по умолчанию `None`): максимальное число категорий, для которых создаются отдельные столбцы. Самые частые `max_categories` категорий кодируются отдельно, остальные - попадают в категорию «редкие» или игнорируются в зависимости от других параметров.

Метод `fit_transform`:

- `fit` находит все уникальные категории в колонке (например, `['ICMP', 'TCP', 'UDP']`);
- `transform` - превращает их в бинарную матрицу (`n_samples`, `n_categories`).

Метод `get_feature_names_out` - возвращает понятные имена признаков после кодирования.

In [ ]:
```
from sklearn.preprocessing import OneHotEncoder
ohe_sklearn = OneHotEncoder(handle_unknown = "ignore", sparse_output = False)
X_ohe_sklearn = ohe_sklearn.fit_transform(df[["protocol"]])

ohe_feature_names = ohe_sklearn.get_feature_names_out(["protocol"])
protocol_ohe_sklearn = pd.DataFrame(X_ohe_sklearn, columns = ohe_feature_names)
protocol_ohe_sklearn['protocol'] = df['protocol']

protocol_ohe_sklearn.head(6)
```

Out[ ]:

	protocol_ICMP	protocol_TCP	protocol_UDP	protocol
0	0.0	1.0	0.0	TCP
1	0.0	1.0	0.0	TCP
2	0.0	0.0	1.0	UDP
3	0.0	1.0	0.0	TCP
4	0.0	0.0	1.0	UDP
5	1.0	0.0	0.0	ICMP

Первый недостаток метода One-Hot Encoding заключается в существенном увеличении объема данных, так как большие по количеству уникальных значений признаки кодируются большим количеством бинарных признаков.

Второй недостаток заключается в том, что если количество новых признаков будет равно числу категорий, то возникает проблема, известная как **ловушка фиктивных переменных**. Когда это произойдет, по крайней мере два новых признака будут мультиколлинеарны, т. е. идеально коррелированы. Это приводит к неверной работе алгоритмов машинного обучения.

В связи с этим количество создаваемых новых признаков должно быть равно $k - 1$, где k — это количество различных категорий признака.

Например, если из полученной таблицы убрать признак "UDP", то при «срабатывании» этого признака (наблюдение с индексом 4) все остальные признаки будут иметь нулевые значения, что позволяет понять, что речь идет именно об этом отсутствующем признаке.

Для удаления одного из новых признаков при создании объекта класса `OneHotEncoder` необходимо использовать параметр `drop`:

- `drop = first` - удаление первой категории признака;
- `drop[i]` - удаление i-й категории признака;
- `if_binary` - удаление первой категории в каждом признаке с двумя категориями.

Необходимо быть осторожным, поскольку применение кодирования для всех категориальных признаков, среди которых есть признак с одной категорией, удаляет этот признак.

In [ ]:
```
ohe = OneHotEncoder(handle_unknown = 'ignore',
                    drop = 'first',
                    sparse_output = False)
X_ohe = ohe.fit_transform(df[["protocol"]])

ohe_feature_names = ohe.get_feature_names_out(["protocol"])
protocol_ohe_sklearn = pd.DataFrame(X_ohe, columns=ohe_feature_names)
protocol_ohe_sklearn['protocol'] = df['protocol']

protocol_ohe_sklearn.head(6)
```

Out[ ]:

	protocol_TCP	protocol_UDP	protocol
0	1.0	0.0	TCP
1	1.0	0.0	TCP
2	0.0	1.0	UDP
3	1.0	0.0	TCP
4	0.0	1.0	UDP
5	0.0	0.0	ICMP

После кодирования необходимо добавить новые признаки к исходному датафрейму, удалив исходный признак.

pd.get_dummies

Альтернативным способом реализации метода One-Hot Encoding является использование метода `get_dummies()` библиотеки `Pandas`.

Этот метод создает новый `DataFrame` со столбцами уникальных категорий и значениями, представляющими наличие этих категорий в строке.

Если применить метод `get_dummies()` к признаку с типом `'object'`, то будут возвращены булевы переменные.

Для перехода к числовым кодам можно либо использовать атрибут `str` при обращении к признаку, либо при вызове функции `get_dummies()` указать тип данных `int`.

Параметр `drop_first = True` позволяет удалить первый закодированный признак для устранения мультиколлинеарности.

Параметр `prefix` позволяет изменять префикс, который добавляется к имени исходной категории.

Параметр `prefix_sep` позволяет изменять разделитель между префиксом и название признака.

Метод автоматически удаляет из таблицы исходный признак.

In [ ]:
```
protocol_dummy = pd.get_dummies(df["protocol"],
                                prefix = "protocol",
                                drop_first = True,
                                dtype = int)

pd.concat([df[["protocol"]], protocol_dummy], axis = 1).head(6)
```

Out[ ]:

	protocol	protocol_TCP	protocol_UDP
0	TCP	1	0
1	TCP	1	0
2	UDP	0	1
3	TCP	1	0
4	UDP	0	1
5	ICMP	0	0

Category Encoders

Метод One-Hot Encoding реализован в классе `OneHotEncoder` библиотеки `Category Encoders`

Основные параметры:

- `cols` (по умолчанию `None`): список признаков для кодирования. Если не задан, кодируются все колонки с типами `object` или `category`;
- `return_df` (по умолчанию `True`): если `True`, возвращается `pandas.DataFrame` с теми же именами колонок; если `False` - `numpy.ndarray`;
- `handle_unknown` (по умолчанию `value`): что делать с категориями, которых не было в обучении:

    - `value` : кодировать как `unknown_value` ;
    - `return_nan` : оставить `NaN` ;
    - `error` : выдать ошибку;
- `unknown_value` (по умолчанию -1): значение для неизвестных категорий (работает при `handle_unknown=value`). В One-Hot это обычно означает, что для новой категории создаётся отдельный бинарный признак (или все признаки будут 0, зависит от реализации; в `category_encoders` чаще делают отдельный «unknown»-столбец).;
- `handle_missing` (по умолчанию `value`) — как обрабатывать пропуски:
    - `value` : заменить на `missing_value` ;
    - `return_nan` : оставить `NaN` ;
    - `error` : выдать ошибку;
    - `indicator` : пропущенные значения будут рассматриваться как отдельная категория, а в обучающем наборе будет добавлен дополнительный фиктивный столбец (независимо от того, есть ли в обучающем наборе пропущенные значения);
- `use_cat_names` (по умолчанию `False`) - ключевой параметр для читаемости:
    - `False` : имена колонок будут вида `col_0` , `col_1` , ...;
    - `True` : имена будут понятными, например `country_RU` , `country_US` .

Важные отличия от sklearn-версии.

Особенность	`category_encoders.OneHotEncoder`	`sklearn.preprocessing.OneHotEnc`
Формат входа/выхода	Принимает и возвращает `pandas.DataFrame`	Возвращает `numpy.array` / `scipy.sparse`
Имена колонок	Автоматически делает понятные имена через `use_cat_names=True`	Нужно вручную вызывать `get_feature_names_out`
Удобство в `Jupyter`	Сразу готов к анализу, корреляциям, графикам	Требует дополнительного шага конвертации в `DataFrame`
Обработка неизвестных	Параметр `handle_unknown` работает аналогично `sklearn`	Аналогично
Интеграция с пайплайнами	Работает в `sklearn.pipeline.Pipeline` , но выход — `DataFrame`	Выход — массив; в `ColumnTransform` удобно собирать матрицу

Пример использования для кодирования признака `protocol` .

In [ ]:
```
import category_encoders as ce
ohe_ce = ce.OneHotEncoder(cols = ['protocol'],
                          # автоматически убирает столбцы, где все значения один
                          drop_invariant = True,
                          use_cat_names = True,
                          handle_unknown = 'indicator')
```

```
X_ohe_ce = ohe_ce.fit_transform(df[['protocol']])
X_ohe_ce.head(6)
```

Out[ ]:

	protocol_TCP	protocol_UDP	protocol_ICMP
0	1	0	0
1	1	0	0
2	0	1	0
3	1	0	0
4	0	1	0
5	0	0	1

Binary Encoding

Binary Encoding — это гибридный метод кодирования категориальных признаков, сочетающий Label Encoding и One-Hot Encoding. Он эффективно сокращает размерность данных при сохранении большей части информации.

Сначала каждой категории присваивается порядковый номер (как в Label Encoding).

Этот номер переводится из 10-й системы в двоичную.

Каждый бит двоичного числа становится отдельным бинарным признаком.

Если количество категорий равно k, то число бинарных признаков определяется как: $\log_2(k)$.

Например для 3-х категорий требуется:

$$\log_2(3) = 2,$$

то есть 2 бинарных столбца вместо 3 столбцов при One-Hot Encoding.

Для 7-ми категорий требуется:

$$[\log_2(7)] = 3,$$

то есть 3 столбца вместо 7.

Ограничения метода.

1. Искусственный порядок категорий. Как и в Label Encoding, категории получают коды, зависящие от сортировки или словаря.
2. Сложная интерпретация. Биты не всегда имеют понятный смысл для аналитика ИБ.
3. Не всегда подходит для линейных моделей. Бинарные биты могут создавать искусственные взаимодействия.

4. Нужен фиксированный словарь. Новые категории нужно уметь кодировать. Обычно для них создают отдельный код или переводят в `unknown`.
5. Потенциальное смешение информации — близкие номера могут иметь сильно отличающиеся двоичные коды (например, 3 → 011, 4 → 100).
6. Не сохраняет семантическую близость — например, две совершенно разные по реализации атаки - «флуд» и «DDoS» - могут получить случайно близкие или далёкие коды.

Рекомендации по использованию в кибербезопасности.

Binary Encoding полезен, если:

- категорий больше, чем 10–20;
- нужно сэкономить память по сравнению с one-hot;
- используется древовидная модель;
- признак не требует высокой интерпретируемости.

Примеры в ИБ:

- имена сервисов;
- категории процессов;
- MITRE-тактики, если их много;
- имена правил нормализации логов.

Не рекомендуется использовать binary encoding для:

- линейных моделей без дополнительной проверки;
- признаков, где важна прозрачность;
- очень высококардинальных признаков (например, IP).

Метод Binary Encoding реализован в классе `BinaryEncoder` библиотеки `Category Encoders`.

Основные параметры:

- `cols` (по умолчанию `None`): список колонок для кодирования. Если не задан, кодируются все колонки с типами `object` или `category`;
- `return_df` (по умолчанию `True`): если `True`, возвращает `pandas.DataFrame` с именами колонок; если `False` - numpy.ndarray;
- `handle_unknown` (по умолчанию `value`): как поступать с категориями, которых не было в обучении:
  - `value`: кодировать как `unknown_value` (неизвестной категории присваивается номер 0 и кодируется как 00...0);
  - `return_nan`: оставить `NaN`;
  - `error`: выдать ошибку;
- `handle_missing` (по умолчанию `value`) — как обрабатывать пропуски:
  - `value`: заменить на `missing_value`;
  - `return_nan`: оставить `NaN`;
  - `error`: выдать ошибку;

- `indicator`: пропущенные значения будут рассматриваться как отдельная категория, а в обучающем наборе будет добавлен дополнительный фиктивный столбец (независимо от того, есть ли в обучающем наборе пропущенные значения).

Рассмотрим пример использования Binary Encoding для кодирования признака `protocol`.

In [ ]:
```
binary_encoder = ce.BinaryEncoder(cols=["protocol"])
protocol_binary_ce = binary_encoder.fit_transform(df["protocol"])

pd.concat([df[["protocol"]], protocol_binary_ce], axis=1).head(6)
```

Out[ ]:

	protocol	protocol_0	protocol_1
0	TCP	0	1
1	TCP	0	1
2	UDP	1	0
3	TCP	0	1
4	UDP	1	0
5	ICMP	1	1

Count / Frequency Encoding

Метод Count Encoding заменяет категорию числом её появлений в данных:
$\mathrm{Count}(k) = n_k$

Метод Frequency Encoding заменяет категорию частотой её встречаемости:
$\mathrm{Freq}(k) = \frac{n_k}{N}$

где:
n_k — число наблюдений в категории k;
n — общее число наблюдений.

В кибербезопасности частота может быть полезным сигналом:

- редкий протокол может быть подозрительным;
- частый сервис может быть нормальным;
- редкий пользовательский процесс может быть аномалией;
- частый IP может быть внутренним сканером или сервером.

Но частота сама по себе не означает атаку.

Ограничения метода.

1. Не учитывает целевую переменную. Частая категория не обязательно связана с атаками.

2. Может усиливать редкие категории. Если редкий признак считается аномальным, Count Encoding может помочь, но может и переобучиться на шум.
3. Зависит от распределения данных. При изменении профиля трафика частоты могут сильно измениться.
4. Новые категории. Для неизвестной категории нужно задавать частоту по умолчанию, обычно 0 или `unknown_freq`.

Методы Count / Frequency Encoding реализованы в классе `CountEncoder` библиотеки `Category Encoders`.

Основные параметры:

- `cols` (по умолчанию `None`): список колонок для кодирования. Если не задан, кодируются все колонки с типами `object` или `category`;
- `return_df` (по умолчанию `True`): если `True`, возвращает `pandas.DataFrame` с именами колонок; если `False` - numpy.ndarray;
- `handle_unknown` (по умолчанию `value`): как поступать с категориями, которых не было в обучении:
  - `value`: кодировать как `unknown_value` (неизвестной категории присваивается номер 0 и кодируется как 00...0);
  - `return_nan`: оставить `NaN`;
  - `error`: выдать ошибку;
- `handle_missing` (по умолчанию `value`) — как обрабатывать пропуски:
  - `value`: заменить на `missing_value`;
  - `return_nan`: оставить `NaN`;
  - `error`: выдать ошибку;
  - `indicator`: пропущенные значения будут рассматриваться как отдельная категория, а в обучающем наборе будет добавлен дополнительный фиктивный столбец (независимо от того, есть ли в обучающем наборе пропущенные значения);
- `min_group_size` (по умолчанию `None`): порог для объединения редких категорий. Может быть:
  - целым числом: минимум столько вхождений, чтобы категория осталась отдельной;
  - дробным числом от 0 до 1: минимальная доля наблюдений. Все категории ниже порога объединяются в одну группу (её имя можно задать через `min_group_name`);
- `min_group_name` (по умолчанию `None`): имя объединённой группы редких категорий. Если `None`, имя формируется автоматически (обычно как конкатенация имён слитых категорий);
- `combine_min_nan_groups` (по умолчанию `True`): объединять ли редкие категории вместе с пропусками в одну группу. Полезно, когда и редкие категории, и пропуски несут похожий смысл «неинформативного/необычного» значения.

Рассмотрим пример использования Count / Frequency Encoding для кодирования признака `service` (имя сервиса).

In [ ]:
```
count_encoder = ce.CountEncoder(cols = ["service"], return_df = True)
service_count_ce = count_encoder.fit_transform(df[["service"]])

service_count_ce = pd.concat([df[["service"]], service_count_ce], axis = 1)
service_count_ce.head(6)
```

Out[ ]:

	service	service
0	ssh	2
1	http	3
2	dns	2
3	smb	1
4	rdp	1
5	none	2

Рекомендации по использованию в кибербезопасности.

Count/Frequency Encoding подходит для:

- признаков сервиса;
- протоколов;
- типов событий;
- категорий процессов;
- правил SIEM;
- признаков, где редкость важна сама по себе.

Feature Hashing

Метод Feature Hashing преобразует категорию в номер корзины с помощью хэш-функции.

Формула:

$$h(k) = \text{hash}(k) \bmod d,$$

где:
k — категория;
d — заранее заданная размерность (число корзин);
$h(k)$ — номер признака-корзины, в которую попадает категория.

Например, при $d = 4$ каждая категория попадёт в одну из четырёх корзин: 0, 1, 2 или 3.

В кибербезопасности Feature Hashing особенно полезен для высококардинальных признаков:

- IP;

- домены;
- URL;
- хэши файлов;
- user-agent;
- идентификаторы сессий.

Главное преимущество: не нужно хранить словарь всех категорий. Это удобно для streaming-систем, SIEM-потоков и online-детекции.

Ограничения метода.

1. Коллизии. Разные категории могут попасть в одну корзину. Например, `UDP` и `ICMP` могут попасть в одну и ту же корзину.
2. Нет обратной интерпретации. По хэш-корзине трудно понять, какая исходная категория туда попала.
3. Сложно объяснять инциденты. В кибербезопасности часто нужно показать конкретный IP или домен, а не «корзину 3».
4. Адверсариальные коллизии. Если злоумышленник знает хэш-функцию и размерность, теоретически он может подбирать значения, попадающие в те же корзины, что и нормальные объекты.
5. Размерность нужно выбирать заранее. Слишком малая размерность увеличивает коллизии, слишком большая уменьшает компактность.

Метод Feature Hashing реализован в классе `HashingEncoder` библиотеки `Category Encoders`.

Основные параметры:

- `cols` (по умолчанию `None`): список колонок для кодирования. Если не задан, кодируются все колонки с типами `object` или `category`;
- `return_df` (по умолчанию `True`): если `True`, возвращает `pandas.DataFrame` с именами колонок; если `False` - numpy.ndarray;
- `n_components` (по умолчанию 8): число выходных бинарных/счётных признаков (размер хеш-пространства). Это ключевой гиперпараметр: чем больше, тем меньше коллизий, но выше размерность. Для высококардинальных признаков часто берут 16–64;
- `hash_method`: какой метод хеширования использовать. Работает любой метод из библиотеки `hashlib`.

Рассмотрим пример использования метода Feature Hashing для кодирования признака `alert_rule` (имя правила, по которому сработало оповещение).

In [ ]:
```
hashing_encoder = ce.HashingEncoder(n_components = 4)
hashing_encoder = hashing_encoder.fit_transform(df[['alert_rule']])

hashing_encoder.head()
```

Out[ ]:

	col_0	col_1	col_2	col_3
0	1	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	0	0
4	0	0	1	0

`col_0`, `col_1`, `col_2`, `col_3` - это новые признаки-корзины (`buckets`), в которые попали категории из колонки `alert_rule` после применения хэширования.

Их количество равно параметру `n_components=4`, то есть мы искусственно создали 4 признака вместо потенциально большого числа уникальных категорий.

Значения 0 и 1 означают индикацию попадания категории в корзину:

- 1 - категория (значение `alert_rule`) попала именно в эту корзину;
- 0 - не попала.

Target Encoding

Этот вид кодировщиков основан на использовании значений (меток) целевого (зависимого) признака для кодирования независимых категориальных признаков.

Target Encoding реализован "под капотом" в модели градиентного бустинга над решающими деревьями в библиотеке `CatBoost`.

Если решается задача регрессии, т. е. построение модели зависимости между вещественным целевым и независимыми признаками, для кодирования используется среднее значение целевого признака, вычисленное для каждой категории категориального признака.

В задаче классификации значения категориального признака кодируются оценками вероятностей целевого признака, вычисленных для каждой категории.

На практике, чтобы уменьшить переобучение на редких категориях, рекомендуется использовать сглаживание (smoothed target encoding):

$$TE_{\text{smooth}}(k) = \frac{n_k \cdot \bar{y}_k + m \cdot \text{prior}}{n_k + m},$$

где:
$\bar{y}_k$ — доля объектов класса 1 (атака) в категории k;
n_k — число событий категории k;
prior — доля атак, вычисленная по все выборке;
m — сила сглаживания.

Пример сглаживания.

Пусть общая доля атак равна: $\text{prior} = 0.1$.

Категория `critical` встретилась только два раза, и оба события оказались атаками: $\bar{y}_{\text{critical}} = 1$.

Тогда без сглаживания: `critical` $\rightarrow 1{,}0$.

Такой результат ненадёжен, поскольку основан только на двух наблюдениях.

После сглаживания может получиться: `critical` $\rightarrow 0{,}24$.

Чем меньше наблюдений в категории, тем сильнее её значение приближается к общей доле атак.

Ограничения метода.

1. Высокий риск того, что модель получит доступ к информации, напрямую или косвенно связанной с целевой переменной (тем, что модель пытается предсказать), чего в реальных условиях просто не будет.
2. Переобучение на редких категориях. Категория с одним событием может получить значение 0 или 1, что нестабильно.
3. Чувствительность к концептуальному дрейфу. Историческая вредоносность IP или страны может измениться.

Для реализации Target Encoding можно использовать несколько подходов:

- "ручное" кодирование на основе механизма `mapping`;
- использование класса `TargetEncoder` модуля `preprocessing` библиотеки `sklearn`;
- использование класса `TargetEncoder` библиотеки `category_encoders`.

mapping

Предположим решается задача классификации - предсказание класса трафика (признак `is_attack`) по значениям независимых признаков. Одним из предикторов является признак `country`.

При использовании Target Encoding для каждой категории признака `country` вычисляется доля наблюдений с вредоносным трафиком (`is_attack = 1`) и полученные значения используются в качестве кодов.

Такой подход позволяет модели "видеть" по значениям вероятности какие типы протокола более часто используются для атаки, чем больше вероятность, тем больше риск атаки.

Реализуем сглаженный Target Encoding.

In [ ]:
```python
# Функция кодирования методом smoothed target encoding

# На вход подаются:
# - series - категориальный признак;
# - y - бинарный целевой признак;
# - m - сила сглаживания;
# - prior - априорная вероятность атаки;
# - min_samples_leaf - минимальное количество наблюдений в категории

def target_encode_smooth(series, y, m=2.0, prior=None, min_samples_leaf=0):
    # Если prior не передан, берётся средняя доля атак по всему датасету
    if prior is None:
        prior = y.mean()

    # Для каждой категори вычисляются:
    # - доля атак внутри категории;
    # - число наблюдений в категории.
    # Применяет сглаживание по формуле

    stats = (pd.DataFrame({"category": series, "y": y})
        .groupby("category")["y"]
        .agg(["mean", "count"]))

    stats["encoded"] = (
        (stats["count"] * stats["mean"] + m * prior) /
        (stats["count"] + m)    )

    # Если в категории слишком мало наблюдений (меньше min_samples_leaf),
    # её принудительно кодируем значением prior.
    # Это защита от сильного шума на очень редких категориях

    if min_samples_leaf > 0:
        stats.loc[stats["count"] < min_samples_leaf, "encoded"] = prior

    # .fillna(prior) нужен для категорий, которых не было в обучении: они получа
    encoded = series.map(stats["encoded"]).fillna(prior)

    return encoded, stats
```

Использование функции для кодирования признака `country` .

In [ ]:
```python
df["country_target_smooth"], country_stats = target_encode_smooth(
    df["country"],
    df["is_attack"],
    m = 2.0)
```

```
country_stats
```

Out[ ]:

category	mean	count	encoded
CN	1.00	3	0.800000
IR	1.00	1	0.666667
KP	1.00	1	0.666667
RU	0.00	3	0.200000
US	0.25	4	0.333333

Sklearn

Реализовать метод кодирования Target Encoding можно через класс `TargetEncoder` модуля `preprocessing` библиотеки `Sklearn`.

`TargetEncoder` применяет более сложную схему сглаживания в отличии от описанной выше. Она включает дополнительные корректировки (например, взвешивание, регуляризацию или другую параметризацию).

Основные параметры:

- `categories (auto)` : категории (уникальные значения) для каждого признака:
  - `auto` : автоматическое определение категорий на основе обучающих данных;
  - `list : categories[i]` : содержит категории, ожидаемые в i-м столбце;
- `target_type (auto)` : тип целевого признака:
  - `auto` : тип цели определяется с помощью `type_of_target` ;
  - `continuous` : непрерывная цель;
  - `binary` : бинарная цель;
  - `multiclass` : многоклассовая цель;
- `smooth (auto)` : сила сглаживания (m). Чем больше - тем сильнее редкие категории «притягиваются» к общему доли (или среднему).

In [ ]:

```
from sklearn.preprocessing import TargetEncoder

te_sklearn = TargetEncoder(target_type = 'binary',
                           smooth = 2.0)

te_sklearn.fit(df[["country"]], df["is_attack"])
country_target_ce_sklearn = te_sklearn.transform(df[["country"]])

country_target_ce_sklearn = pd.Series(country_target_ce_sklearn.flatten())

country_target_ce_sklearn = pd.concat([df[["country"]],
                                       country_target_ce_sklearn], axis=1)
country_target_ce_sklearn.head(6)
```

Out[ ]:

	country	0
0	CN	0.800000
1	US	0.333333
2	RU	0.200000
3	CN	0.800000
4	KP	0.666667
5	CN	0.800000

Category Encoders

Метод Target Encoding реализован в классе `TargetEncoder` библиотеки `Category Encoders`.

`TargetEncoder` применяет сглаживание, реализуя формулу, описанную выше.

Основные параметры:

- `cols`: список колонок для кодирования, если `None` - все категориальные;
- `return_df` (по умолчанию `True`): если `True`, возвращает `pandas.DataFrame` с именами колонок; если `False` - numpy.ndarray;
- `categories (auto)`: категории (уникальные значения) для каждого признака:
  - `auto`: автоматическое определение категорий на основе обучающих данных;
  - `list : categories[i]`: содержит категории, ожидаемые в i-м столбце;
- `target_type (auto)`: тип целевого признака:
  - `auto`: тип цели определяется с помощью `type_of_target`;
  - `continuous`: непрерывная цель;
  - `binary`: бинарная цель;
  - `multiclass`: многоклассовая цель;
- `smoothing (по умолчанию 1.0)`: сила сглаживания (m). Чем больше - тем сильнее редкие категории «притягиваются» к общему доли (или среднему);
- `min_samples_leaf (по умолчанию 1)`: минимальное число объектов в категории, чтобы использовать её локальную статистику. Если меньше — используется `prior`;
- `handle_unknown`: что делать с неизвестными категориями на тесте:
  - `error` - выдать ошибку;
  - `ignore` - заменить на `prior`;
- `handle_missing` (по умолчанию `value`) — как обрабатывать пропуски:
  - `value`: заменить на `missing_value`;
  - `return_nan`: оставить `NaN`;
  - `error`: выдать ошибку.

In [ ]:
```
target_encoder = ce.TargetEncoder(cols=["country"],
                                  smoothing=1.0,
```

```
                                    min_samples_leaf=2)

target_encoder.fit(df[["country"]], df["is_attack"])
country_target_ce = target_encoder.transform(df[["country"]])

country_target_ce = pd.concat([df[["country"]],
                               country_target_ce], axis=1)

country_target_ce.head(6)
```

Out[ ]:

	country	country
0	CN	0.865529
1	US	0.279801
2	RU	0.134471
3	CN	0.865529
4	KP	0.634471
5	CN	0.865529

Рекомендации по использованию в кибербезопасности.

Target Encoding подходит для кодирования:

- стран;
- ASN;
- IP-репутаций;
- типов процессов;
- правил SIEM с историей подтвержденных инцидентов.

Рекомендации:

- всегда использовать сглаживание;
- считать статистики только на `train`;
- задавать минимальный порог выборки;
- для новых категорий использовать `prior`;
- периодически обновлять статистики;

В кибербезопасности Target Encoding может быть сильным признаком, но при неправильном использовании он часто приводит к утечке и переобучению.

Leave-One-Out Target Encoding

Leave-One-Out Target Encoding — это вариант метода Target Encoding, при котором для каждого наблюдения целевая статистика считается по той же категории, но без текущего наблюдения.

Формула:

$$LOO_i = \frac{\sum_{j \in k_i,\, j \neq i} y_j}{n_{k_i} - 1},$$

где:
k_i — категория текущего события;
n_{k_i} — число событий этой категории;
y_j — целевая переменная события j.

Если категория встречается один раз, то оценка не может быть рассчитана. В этом случае обычно используется глобальный `prior`.

Идея метода: уменьшить прямую утечку целевой переменной, исключая текущий объект из статистики.

Метод Leave-One-Out Target Encoding реализован в классе `LeaveOneOutEncoder` библиотеки `Category Encoders`.

Основные параметры:

- `cols`: список колонок для кодирования, если `None` - все категориальные;
- `return_df` (по умолчанию `True`): если `True`, возвращает `pandas.DataFrame` с именами колонок; если `False` - numpy.ndarray;
- `handle_unknown`: что делать с неизвестными категориями на тесте:
  - `error` - выдать ошибку;
  - `ignore` - заменить на `prior`;
- `handle_missing` (по умолчанию `value`) — как обрабатывать пропуски:
  - `value`: заменить на `missing_value`;
  - `return_nan`: оставить `NaN`;
  - `error`: выдать ошибку;
- `sigma` (по умолчанию `None`): уровень добавляемого шума (нормальный шум с нулевым средним и дисперсией `sigma`). Нужен, чтобы дополнительно «размыть» кодированные значения и снизить риск переобучения. Если `None`, шум не добавляется. В задачах кибербезопасности (редкие типы атак) небольшой шум может быть полезен.

Пример кодирования признака `protocol`.

In [ ]:
```
loo_encoder = ce.LeaveOneOutEncoder(cols = ["protocol"])
loo_encoder.fit(df[["protocol"]], df["is_attack"])

protocol_loo_ce = loo_encoder.transform(df[["protocol"]])

protocol_loo_ce.head()
```

Out[ ]:

	protocol
0	0.666667
1	0.666667
2	0.250000
3	0.666667
4	0.250000

Ordered Target Encoding / CatBoost-подобное кодирование

Ordered Target Encoding использует порядок событий. Для каждого события кодирование считается только по предыдущим событиям той же категории.

Это особенно естественно для кибербезопасности, потому что события приходят во времени:

- сначала лог, потом детекция;
- сначала обучение, потом эксплуатация;
- сначала исторические инциденты, потом новые события.

Формула для Ordered Target Encoding (OTE).

Если для категории k до текущего момента уже есть n_{prev} событий и сумма их целевых значений S_{prev}, то:

$$OTE_i = \frac{S_{\text{prev}} + m \cdot \text{prior}}{n_{\text{prev}} + m}.$$

Если предыдущих событий нет:

$$OTE_i = \text{prior},$$

где:
prior — глобальная доля атак;
m — параметр сглаживания.

Такой подход уменьшает целевую утечку, потому что не использует будущие события.

Ограничения метода.

1. Зависимость от порядка. Если строки перемешаны, Ordered Target Encoding может потерять смысл. Нужно сортировать по времени.
2. Холодный старт. Для новых категорий нет истории, поэтому используется `prior`.

3. Может недооценивать редкие категории. Если категория появляется редко, статистика накапливается медленно.
4. Нужно хранить состояние. Для streaming-систем нужно обновлять статистики по каждой категории.
5. Чувствительность к концептуальному дрейфу. Старая статистика может устаревать.

Рассмотрим пример использования Ordered Target Encoding для кодирования признака `protocol`.

Метод Ordered Target Encoding реализован в классе `CatBoostEncoder` библиотеки `Category Encoders`.

`a` - параметр сглаживания.

In [ ]:
```
catboost_encoder = ce.CatBoostEncoder(cols = ["protocol"], a = 1.0)

catboost_encoder.fit(df[["protocol"]], df["is_attack"])
protocol_catboost_ce = catboost_encoder.transform(df[["protocol"]])

protocol_catboost_ce.head(6)
```

Out[ ]:

	protocol
0	0.642857
1	0.642857
2	0.300000
3	0.642857
4	0.300000
5	0.500000

Практические рекомендации по выбору метода кодирования номинальных признаков

Небольшое количество категорий (до 5).

Например:

- `protocol` - сетевой протокол, который использовался в событии;
- `direction` - направление сетевого взаимодействия: `inbound` (входящий трафик — извне внутрь сети) или `outbound` (исходящий — из сети наружу);
- `event_type` - тип события: `login_attempt` (попытка входа), `file_download` (скачивание файла), `port_scan` (сканирование портов), `malware_execution` (запуск вредоносного ПО), `access_violation` (нарушение прав доступа) и т.п;
- `severity` - уровень серьёзности события (если она рассматривается как номинальная).

Рекомендуемы методы кодирования:

- One-Hot Encoding;
- Count/Frequency Encoding;
- Target Encoding с регуляризацией.

Большое количество категорий.

Например:

- `src_ip` - IP-адрес устройства, которое инициировало сетевое соединение;
- `dst_ip` - IP-адрес, куда направлен трафик;
- `domain` - доменная часть URL;
- `url` - полный адрес ресурса;
- `file_hash` - криптографическая хеш-сумма файла, которая однозначно его идентифицирует;
- `user_agent` - строка, которую клиент отправляет серверу, чтобы сообщить о себе.

Рекомендуемые методы кодирования:

- Feature Hashing;
- Count/Frequency Encoding;
- Ordered Target Encoding;
- Target Encoding с сильным сглаживанием и минимальным порогом выборки.

Присутствует временной порядок событий.

Например:

- SIEM-логи;
- `netflow` - агрегированные данные о сетевых потоках;
- `authentication_events` - события аутентификации;
- DNS-запросы;
- `alert_history` - история алертов (прошлые срабатывания системы обнаружения вторжений).

Рекомендуемые методы кодирования:

- Ordered Target Encoding.

При использовании линейной модели с необходимостью интерпретации влияния независимых признаков.

Например:

- логистическая регрессия;
- линейный SVM.

Рекомендуемые методы кодирования:

- One-Hot Encoding;
- Сглаженный Target Encoding.

При использовании моделей, основанных на деревьях решений.

Например:

- Random Forest;
- Gradient Boosting;
- XGBoost;
- LightGBM;
- CatBoost.

Рекомендуемые методы кодирования:

- Label Encoding;
- Binary Encoding;
- Target Encoding;
- Count/Frequency Encoding;
- Feature Hashing.

2. Кодирование порядковых признаков

Порядковый признак — это категориальный признак, у которого значения имеют естественный порядок, но расстояния между уровнями обычно неизвестны и не обязательно равны.

Примеры порядковых признаков в кибербезопасности:

- уровень критичности алерта: `low`, `medium`, `high`, `critical`;
- уровень привилегий пользователя: `user`, `operator`, `admin`, `root`;
- диапазон CVSS: `0-3.9`, `4-6.9`, `7-8.9`, `9-10`;
- число неудачных входов: `0`, `1-2`, `3-5`, `>5`;
- приоритет инцидента: `P4`, `P3`, `P2`, `P1`.

При рассмотрении технологий кодирования номинальных признаков было показано, что по умолчанию катгории признака сортируются по алфавитному порядку.

Такая ситуация не приемлема для порядковых признаков - для них порядок нужно задавать явно.

Продолжим работу с датасетом из предыдущего вопроса. Добавим в него порядковые признаки, характерные для SOC/ИБ-аналитики:

- `severity` — критичность события;
- `confidence` — уверенность правила или модели детекции;

- `asset_criticality` — критичность актива, которого касается событие.

In [ ]:
```python
# @title
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df["severity"] = [
    "high", "low", "low", "critical",
    "high", "medium", "critical", "low",
    "high", "low", "medium", "low"
]

df["confidence"] = [
    "high", "low", "low", "high",
    "medium", "medium", "high", "low",
    "medium", "low", "medium", "low"
]

df["asset_criticality"] = [
    "critical", "public", "internal", "critical",
    "important", "internal", "critical", "public",
    "important", "internal", "public", "public"
]

ordinal_cols = ["severity", "confidence", "asset_criticality"]

severity_order = ["low", "medium", "high", "critical"]
confidence_order = ["low", "medium", "high"]
asset_order = ["public", "internal", "important", "critical"]

ordinal_orders = {
    "severity": severity_order,
    "confidence": confidence_order,
    "asset_criticality": asset_order
}
```

In [ ]:
```python
df[["severity", "confidence", "asset_criticality", "is_attack"]].head()
```

Out[ ]:

	severity	confidence	asset_criticality	is_attack
0	high	high	critical	1
1	low	low	public	0
2	low	low	internal	0
3	critical	high	critical	1
4	high	medium	important	1

Label Encoding

Самый естественный способ закодировать порядковый признак — заменить категории числами так, чтобы порядок чисел соответствовал порядку категорий.

Формально:

$$x_i \to \text{code}(x_i),$$

где, если категория a меньше категории b, то:

$$\text{code}(a) < \text{code}(b)$$

Для реализации этого метода кодирования можно использовать те же подходы, что и для номинальных признаков:

- "ручное" кодирование на основе механизма `mapping`;
- преобразование признака к типу `'category'` и использование метода `cat.codes()`;
- использование классов `LabelEncoder` или `OrdinalEncoder` модуля `preprocessing` библиотеки `sklearn`;
- использование класса `OrdinalEncoder` библиотеки `category_encoders`.

Рассмотрим пример кодирования признака `severity` (критичность события).

В `sklearn.preprocessing` для кодирования порядковых признаков методом Label Encoding используется класс `OrdinalEncoder`.

In [ ]:
```
from sklearn.preprocessing import OrdinalEncoder

feature = "severity"

# Указание явного порядка категорий
# Это важно: именно так кодировщик поймёт, что Low < Medium < High < Critical
severity_order = ["low", "medium", "high", "critical"]

# список категорий
categories_list = [severity_order]

# handle_unknown="use_encoded_value" и unknown_value=-1:
# если встретится категория, которой не было в обучении
# (например, новый уровень критичности),
# ей присвоится значение -1. Рекомендуется в кибербезопасности:
# новые/неизвестные уровни сразу видно по отрицательному коду

# categories должен быть списком списков: [[категории для признака 1]]

oe_sklearn = OrdinalEncoder(categories = categories_list,
                            handle_unknown = "use_encoded_value",
                            unknown_value = -1)

X_oe_sklearn = oe_sklearn.fit_transform(df[[feature]])

# Создание DataFrame с понятной колонкой
ordinal_df_sklearn = pd.DataFrame(X_oe_sklearn,
                                  columns = [f"{feature}_sklearn"])

# Объединение исходной и закодированной колонки
result_sklearn = pd.concat([df[[feature]], ordinal_df_sklearn], axis=1)

result_sklearn.head()
```

Out[ ]:

	severity	severity_sklearn
0	high	2.0
1	low	0.0
2	low	0.0
3	critical	3.0
4	high	2.0

В качестве альтернативы для кодирования методом Label Encoding можно использовать класс `OrdinalEncoder` библиотеки `Category Encoders`.

Главное отличие от `sklearn` в том, что здесь порядок категорий задается не через `categories`, а через параметр `mapping` (список словарей), а нумерация категорий по умолчанию начинается с 1 (параметр `index_start`).

In [ ]:
```
feature = "severity"

# Явный порядок категорий: ключ → желаемый код
# В category_encoders это передаётся как список словарей в параметре mapping
severity_mapping = [{"col": feature,
                     "mapping": {
                     "low": 0,
                     "medium": 1,
                     "high": 2,
                     "critical": 3}}]

# Создаём энкодер:
# - mapping задаёт жёсткий порядок
# - handle_unknown='value' + handle_missing='value' позволяют контролировать неи
# - index_start=0, чтобы ранги начинались с 0 (по умолчанию с 1)
oe_ce = ce.OrdinalEncoder(
    mapping = severity_mapping,
    handle_unknown = "value",
    handle_missing = "value",
    index_start = 0
)

# fit_transform() возвращает DataFrame, в котором коды записаны в колонку severi
X_oe_ce = oe_ce.fit_transform(df[[feature]])

# Переименование колонки для получения понятного названия
ordinal_df_ce = X_oe_ce.rename(columns = {feature: f"{feature}_ce"})

# Объединение исходной и закодированной колонки
result_ce = pd.concat([df[[feature]], ordinal_df_ce], axis=1)

result_ce.head()
```

Out[ ]:

	severity	severity_ce
0	high	2
1	low	0
2	low	0
3	critical	3
4	high	2

Ограничения метода.

1. Предполагается равный интервал между уровнями. Коды 0, 1, 2, 3 не гарантируют, что переход от `high` к `critical` столь же значителен, как от `low` к `medium`.
2. Неизвестные категории требуют политики обработки. Нельзя просто игнорировать новые значения, особенно в кибербезопасности.
3. Нужна синхронизация словарей. Если в обучающей и тестовой выборках используются разные порядки, возникнет ошибка.

One-Hot Encoding

One-Hot Encoding обычно применяется к номинальным признакам, категории которых не имеют естественного порядка. Однако технически этот метод можно использовать и для порядковых категориальных признаков.

Рассмотрим ситуации, когда One-Hot Encoding оправдан.

Будем в качестве примера использовать признак `severity` (критичность события).

1. Порядок существует, но расстояния между категориями неизвестны.

Для признака `severity`: `low < medium < high < critical`.

Понятно, что уровень `critical` выше чем `high`, но нельзя утверждать, что числовые расстояния между соседними уровнями одинаковы.

One-Hot Encoding не навязывает модели искусственную шкалу.

2. Влияние категорий может быть немонотонным.

Иногда вероятность атаки не увеличивается последовательно с ростом категории.

Например, злоумышленник может специально создавать большое количество событий уровня `medium`, чтобы не вызвать критическое предупреждение. Тогда именно категория `medium`, а не `critical`, может быть наиболее важным индикатором атаки.

В таком случае One-Hot Encoding позволяет модели выявить индивидуальное влияние каждого уровня.

3. Используется линейная модель.

Для линейной или логистической регрессии порядковое кодирование часто задаёт слишком сильное предположение: "чем больше код категории, тем сильнее её влияние".

Однако, наличие порядка не означает, что влияние категорий на целевую переменную обязательно должно быть линейным или монотонным.

Например, при использовании порядкового кодирования признака `severity` было получено:

- `low → 0`;
- `medium → 1`;
- `high → 2`;
- `critical → 3`.

Если использовать закодированные таким образом признаки в линейной модели, то она может интерпретировать переход от `low` к `medium` так же, как переход от `high` к `critical`. Фактически модель предполагает одинаковое расстояние между соседними уровнями. В реальной задаче кибербезопасности это предположение в большинстве случаев не выполняется.

В такой ситуации One-Hot Encoding позволяет модели отдельно определить влияние каждого уровня:

$$z = \beta_0 + \beta_1 \cdot I_{medium} + \beta_2 \cdot I_{high} + \beta_3 \cdot I_{critical}.$$

Здесь коэффициенты категорий не обязаны увеличиваться последовательно. Например, события средней критичности могут оказаться более характерными для конкретной атаки, чем события с меткой `high`.

Для реализации One-Hot Encoding можно использовать рассмотренные выше классы `OneHotEncoder` бибилиотек `sklearn` и `category_encoders`.

Реализация в `sklearn`.

In [ ]:
```
from sklearn.preprocessing import OneHotEncoder

feature = "severity"

# Создаём One-Hot Encoder
ohe_sklearn = OneHotEncoder(categories = [["low", "medium", "high", "critical"]]
                            handle_unknown = "ignore",
                            sparse_output = False,
                            dtype = int)
# Кодируем признак
encoded_values = ohe_sklearn.fit_transform(df[[feature]])
```

```
# Создаём DataFrame с закодированными значениями
encoded_df = pd.DataFrame(encoded_values,
                          columns = ohe_sklearn.get_feature_names_out([feature]),
                          index = df.index)

# Объединяем исходный и закодированные признаки
result_df = pd.concat([df[[feature]], encoded_df], axis = 1)

result_df.head()
```

Out[ ]:

	severity	severity_low	severity_medium	severity_high	severity_critical
0	high	0	0	1	0
1	low	1	0	0	0
2	low	1	0	0	0
3	critical	0	0	0	1
4	high	0	0	1	0

Реализация в `category_encoders`.

In [ ]:

```
import category_encoders as ce

feature = "severity"

# Создаём One-Hot Encoder
ohe_ce = ce.OneHotEncoder(cols = [feature],
                          use_cat_names = True,
                          handle_unknown = "value",
                          handle_missing = "value",
                          return_df = True)
# Кодируем признак
encoded_df_ce = ohe_ce.fit_transform(df[[feature]])

# Объединяем исходный признак с закодированными значениями
result_df_ce = pd.concat([df[[feature]], encoded_df_ce], axis = 1)

result_df_ce.head()
```

Out[ ]:

	severity	severity_high	severity_low	severity_critical	severity_medium
0	high	1	0	0	0
1	low	0	1	0	0
2	low	0	1	0	0
3	critical	0	0	1	0
4	high	1	0	0	0

Thermometer Encoding

Thermometer Encoding — метод кодирования порядковых категориальных признаков, при котором каждый следующий уровень включает признаки всех предыдущих уровней. Метод также называется:

- cumulative encoding;
- rank-hot encoding;
- термометрическим кодированием.

Название связано с тем, что код увеличивается постепенно, подобно столбику термометра.

Принцип кодирования.

Рассмотрим порядковый признак "everity" (критичность события):

`low < medium < high < critical`.

Thermometer Encoding формирует следующую матрицу:

severity	medium_or_higher	high_or_higher	critical_or_higher
low	0	0	0
medium	1	0	0
high	1	1	0
critical	1	1	1

Каждая колонка означает достижение определённого порога:

- `medium_or_higher` — уровень не ниже `medium`;
- `high_or_higher` — уровень не ниже `high`;
- `critical_or_higher` — уровень не ниже `critical`.

Для категории с рангом r значение порогового признака i определяется как:

$$x_i = \begin{cases} 1, & r \geq i \\ 0, & r < i \end{cases}$$

Если исходный признак содержит k уровней, обычно создаётся $k - 1$ полезных бинарных признаков.

Отличие от других методов.

Сравнение с Ordinal Encoding.

Ordinal Encoding формирует один числовой признак:

severity	Код
low	0
medium	1

severity	Код
high	2
critical	3

Для линейной модели это может создать предположение, что влияние каждого перехода одинаково:

$$\text{medium} - \text{low} = \text{high} - \text{medium} = \text{critical} - \text{high}.$$

Thermometer Encoding позволяет назначить каждому переходу отдельный коэффициент:

$$z = \beta_0 + \beta_1 \cdot I(r \geq medium) + \beta_2 \cdot I(r \geq high) + \beta_3 \cdot I(r \geq critical).$$

Здесь:

- β_1 отражает влияние перехода от `low` к `medium`;
- β_2 — дополнительное влияние перехода к `high`;
- β_3 — дополнительное влияние перехода к `critical`.

Таким образом, влияние переходов не обязано быть одинаковым.

Сравнение с One-Hot Encoding.

One-Hot Encoding рассматривает категории независимо:

severity	low	medium	high	critical
low	1	0	0	0
medium	0	1	0	0
high	0	0	1	0
critical	0	0	0	1

Из такого представления модель не видит, что `high` ближе к `critical`, чем к `low`.

Thermometer Encoding сохраняет эту связь:

severity	medium_or_higher	high_or_higher	critical_or_higher
low	0	0	0
medium	1	0	0
high	1	1	0
critical	1	1	1

Соседние уровни отличаются только одним разрядом.

Рекомендации по применению Thermometer Encoding:

- категории имеют однозначный естественный порядок;
- количество уровней относительно невелико;
- расстояния между соседними уровнями неизвестны;
- влияние переходов между уровнями может быть различным;
- используется линейная или логистическая регрессия;
- необходимо сохранить порядок без сведения признака к одному числу.

В библиотеке `category_encoders` метод реализован классом `RankHotEncoder`.

In [34]:
```
import category_encoders as ce

feature = "severity"

# Явно задаём правильный порядок категорий
severity_order = ["low","medium","high","critical"]

# Создаём копию признака
X_severity = df[[feature]].copy()

# Преобразуем строковый признак в упорядоченную категорию.
# Это важно, чтобы RankHotEncoder не использовал
# алфавитный порядок строк.
X_severity[feature] = pd.Categorical(X_severity[feature],
                                     categories = severity_order,
                                     ordered = True)

# Создаём энкодер
rankhot_encoder = ce.RankHotEncoder(cols = [feature],
                                    use_cat_names = True,
                                    drop_invariant = True,
                                    handle_unknown = "error",
                                    handle_missing = "error",
                                    return_df = True)

# Обучаем энкодер и преобразуем признак
encoded_df = rankhot_encoder.fit_transform(X_severity)

# Объединяем исходный признак и закодированные значения
result_df = pd.concat([df[[feature]], encoded_df], axis=1)

result_df.head()
```

Out[34]:

	severity	severity_medium	severity_high	severity_critical
0	high	1	1	0
1	low	0	0	0
2	low	0	0	0
3	critical	1	1	1
4	high	1	1	0

Использование аргумента `drop_invariant=True`.

Внутренне `RankHotEncoder` создаёт колонку и для минимальной категории:

severity	severity_low	severity_medium	severity_high	severity_critical
low	1	0	0	0
medium	1	1	0	0
high	1	1	1	0
critical	1	1	1	1

Колонка `severity_low` равна единице для всех известных категорий. Она не содержит полезной информации, поэтому `drop_invariant=True` удаляет её автоматически.

Target Encoding

Особенность применения к порядковым признакам.

Target Encoding можно применять к порядковым признакам, но метод не сохраняет их естественный порядок автоматически.

Например, исходный порядок признака `severity`:

`low < medium < high < critical`.

После кодирования может превратиться в:

- `low → 0,03`;
- `medium → 0,22`;
- `high → 0,15`;
- `critical → 0,71`.

Значение `medium` оказалось выше `high`, поскольку в обучающей выборке события `medium` чаще были связаны с атаками.

Следовательно, Target Encoding отражает статистическую связь категории с целевой переменной, а не естественный ранг категории.

Если частота атак увеличивается вместе с уровнем критичности, порядок сохранится косвенно. Но метод не гарантирует ни упорядоченность, ни монотонность полученных значений.

Когда Target Encoding оправдан.

Метод можно использовать, если:

- категории действительно связаны с целевой переменной;
- влияние категорий может быть немонотонным;
- порядковый признак содержит много категорий;
- One-Hot Encoding слишком сильно увеличивает размерность;
- требуется получить один числовой признак;

- в каждой категории достаточно обучающих наблюдений;
- применяется корректное сглаживание.

Для реализации Target Encoding можно использовать рассмотренные выше классы `OneHotEncoder` бибилиотек `sklearn` и `category_encoders`.

Рассмотрим пример кодирования признака `severity` с использованием библиотеки `category_encoders`.

In [35]:
```
feature = "severity"
target = "is_attack"

# Создаём энкодер
target_encoder = ce.TargetEncoder(cols = [feature],
                                  min_samples_leaf = 20,
                                  smoothing = 10,
                                  handle_unknown = "value",
                                  handle_missing = "value",
                                  return_df = True)
# Обучаем энкодер и преобразуем признак
encoded_df = target_encoder.fit_transform(df[[feature]],
                                          df[target])

# Переименовываем колонку
encoded_df = encoded_df.rename(
    columns={feature: f"{feature}_target"})

# Объединяем исходный признак, целевую переменную
# и результат кодирования
result_df = pd.concat([df[[feature, target]], encoded_df], axis = 1)

result_df.head(6)
```

Out[35]:

	severity	is_attack	severity_target
0	high	1	0.525744
1	low	0	0.408787
2	low	0	0.408787
3	critical	1	0.570926
4	high	1	0.525744
5	medium	1	0.570926

Для анализа результата удобно использовать сводную таблицу, в которой для каждой категории вывести количество наблюдений, исходную долю атак и сглаженное значение.

In [36]:
```
severity_order = ["low", "medium", "high", "critical"]

summary_df = (
    result_df
    .groupby(feature, observed=True)
    .agg(
```

```
        count=(target, "size"),
        attack_rate=(target, "mean"),
        target_encoded=(f"{feature}_target", "first")
    )
    .reindex(severity_order)
    .reset_index()
)

summary_df
```

Out[36]:

	severity	count	attack_rate	target_encoded
0	low	5	0.000000	0.408787
1	medium	2	1.000000	0.570926
2	high	3	0.666667	0.525744
3	critical	2	1.000000	0.570926

Для редкой категории `critical` закодированное значение значительно отличается от доли атак для этой категории (1) из-за сглаживания.

Литература

1. Криволапов, С. Я. Анализ данных. Методы теории вероятностей и математической статистики на языке Python : учебное пособие / С.Я. Криволапов. – Москва : ИНФРА-М, 2025. – 678 с.
2. Груздев А.В. Предварительная подготовка данных в Python. Том 2: План, примеры и метрики качества. – М.: ДМК Пресс, 2023. – 814 с.
3. Демидова, Л. А. Разведочный анализ данных. Python : учебно-методическое пособие / Л. А. Демидова. — Москва : РТУ МИРЭА, 2022 — Часть 1 — 2022. — 107 с.
4. Демидова, Л. А. Разведочный анализ данных. Python : учебно-методическое пособие / Л. А. Демидова. — Москва : РТУ МИРЭА, 2023 — Часть 2 — 2023. — 92 с.